MBLP131 · Hafta 11 · Ders Notları

İşletim Sistemleri Temelleri

İzinler, Sahiplik ve Kullanıcılar

Dosyalarınızı kim okuyabilir, değiştirebilir, çalıştırabilir — ve Linux buna nasıl karar verir?

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. Bir işletim sisteminin neden dosya izinlerine ihtiyaç duyduğunu ve çekirdeğin bunları nasıl uyguladığını açıklamak (2. ve 7. haftalarla bağlantı).
2. Kullanıcı adınızı, UID'nizi, GID'nizi ve gruplarınızı `whoami`, `id` ve `groups` ile bulmak; `/etc/passwd` ve `/etc/group` alanlarını okumak.
3. `-rwxr-x---` gibi bir mod dizgesini çözmek, sekizliye (750) ve geri dönüştürmek.
4. r, w ve x'in dosyalar *ve* dizinler için ne anlama geldiğini; sahip, grup ve diğerleri sınıflarından hangisinin bir sürece uygulandığını açıklamak.
5. İzinleri `chmod` ile sembolik (`u+x`, `go-w`) ve sekizli (`640`) biçimde değiştirmek; bir betiği çalıştırılabilir yapmak.
6. Varsayılan izinleri `umask` değerinden doğru hesaplamak (çıkarmayla değil, bit düzeyinde).
7. `chown`, `chgrp`, `sudo`, `passwd`'nin setuid bitini ve `/tmp`'nin yapışkan bitini açıklamak; ilgili hata iletilerini okumak.

İçindekiler

1 10. Haftadan 11. Haftaya 1

2 Neden İzinler? Korumaya Yeniden Bakış 2

3 Kullanıcılar ve Gruplar 3

4 İzinleri Okumak 4

5 İzinleri `chmod` ile Değiştirmek 6

6 Varsayılan İzinler: `umask` 9

7 Uygulamada Dizin İzinleri 10

8 Sahiplik: chown ve chgrp 12

9 root ve sudo 13

10 Özel Bitler: setuid ve Yapışkan Bit 14

11 Komut Başvuru Tablosu 15

12 Sık Görülen Hatalar ve Anlamları 15

13 Özet 16

14 Sözlük 16

15 Laboratuvar Föyü: Bir Proje Klasörünü Güvenli Hâle Getirmek 17

16 Tekrar Soruları 18

17 İleriye Bakış: 12. Hafta 19

18 Laboratuvar Föyü Çözümleri 19

1 10. Haftadan 11. Haftaya

Geçen hafta dosya oluşturdunuz, kopyaladınız, taşıdınız ve sildiniz (`mkdir`, `touch`, `cp`, `mv`, `rm`); joker karakterleri kabuğa genişlettiniz ve dosyaların içine `cat`, `less`, `head`, `tail` ve `stat` ile baktınız. Her `ls -l` ve `stat` çıktısında, bir sahip ve bir grup adıyla birlikte `-rw-rw-r--` ve `Access: (0664/-rw-rw-r--)` gibi bir şey göründü. Bu hafta bu alanları okumayı, değiştirmeyi ve arkalarındaki kullanıcıları ve grupları anlamayı öğreniyoruz.

Örnekleri nasıl izlemeli? Önce alıştırma klasörünü yeniden oluşturun (betik yalnızca `~/lab` klasörünü silip yeniden oluşturur):

```
$ bash mcmp117_lab_setup.sh
```

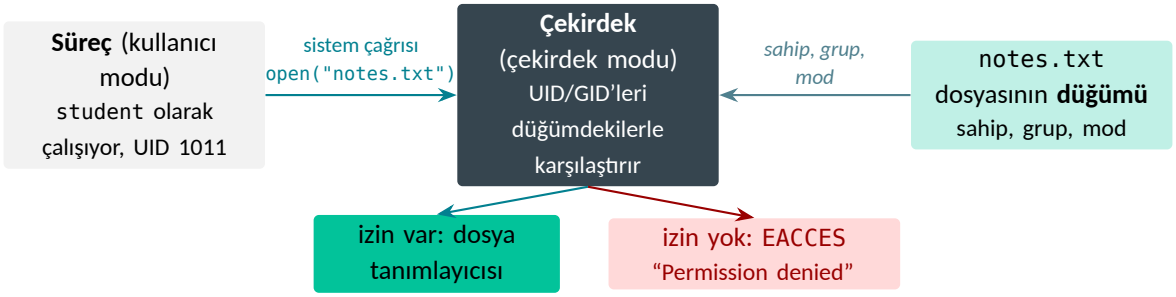
Bu notlardaki her komut, bir Linux makinesinde `student` adlı *sıradan* bir kullanıcıyla (makine adı `okan-lab`) çalıştırıldı; `code/demo_*.sh` betikleri komutların tam hâlini, `code/out_*.txt` dosyaları ise yakalanan çıktıyı içerir. Kendi makinenizde tarihler, boyutlar, kullanıcı ve grup *numaraları* ve adları farklı olacaktır; `ls` çıktısının sırası da yerel ayara (locale) bağlı olabilir (biz `LC_ALL=C.UTF-8` kullandık). Türkçe yerel ayarlı bir sistemde (`LANG=tr_TR.UTF-8`) hata iletilerinin çoğu Türkçe görünür (örneğin *Permission denied* yerine *Erişim engellendi*) ve sıralama farklı olabilir; ders çıktıları bu yüzden İngilizcedir. Yalnızca konu `sudo` ya da `root` olduğunda bir yöneticinin bir şeyi önceden hazırladığını açıkça belirtiyoruz.

2 Neden İzinler? Korumaya Yeniden Bakış

2.1 Arkasındaki kuram

2. haftada ikili mod (dual-mode operation) çalışmasını gördük: işlemci kullanıcı programlarını, doğrudan G/Ç gibi ayrıcalıklı komutların yasak olduğu *kullanıcı modunda* çalıştırır. Bir dosyayı okumak isteyen program diskle kendisi konuşamaz; çekirdek moduna geçiren `open()` gibi bir **sistem çağrısı** (system call) yapmak zorundadır. **7. haftada** *korumayı* (protection) bir dosyanın öznitelikleri arasında saydık ve Linux'un bunu sahip, grup, boyut ve zaman damgalarıyla birlikte dosyanın **düğümünde** (inode) sakladığını gördük.

Bu iki olguyu bir araya getirdiğinizde bütün düzenek ortaya çıkar:



Şekil 1: Her erişim bir sistem çağrısından geçer; bu yüzden çekirdek onu denetleyebilir ve denetim atlanamaz.

Program çekirdekten *istemek zorunda* olduğu için çekirdek reddedebilir. Denetim başarısız olursa sistem çağrısı -1 döndürür ve hata kodunu EACCES yapar; programlar bunu *Permission denied* (izin reddedildi) olarak yazdırır. Aynı denetim bir program çalıştırdığınızda (`exec`), bir dizine girdiğinizde (`chdir`) ya da bir adı sildiğinizde (`unlink`) de yapılır. Denetimin dosya *açılırken* yapıldığına dikkat edin: bir dosyayı zaten açmış olan süreç, izinler sonradan değişse bile erişimini korur.

> **Tarihsel not**
>
> 1969'dan itibaren Bell Laboratuvarları'nda tasarlanan UNIX, en başından çok kullanıcılı bir zaman paylaşımlı sistemdi (1. hafta); bu yüzden ucuz bir koruma düzenine ihtiyaç duyuyordu. Ken Thompson ve Dennis Ritchie, her dosya için kullanıcıların tam bir listesi (bir *erişim denetim listesi*, access control list) yerine her biri üç bitlik üç sınıf seçti — sahip, grup, diğerleri: düğümde dokuz bit. Bu düzen o kadar basit ve etkiliydi ki Linux'ta ve macOS'ta değişmeden yaşıyor. (Günümüz Linux'u `getfacl/setfacl` ile ACL'leri de destekler, ama temel hâlâ dokuz bittir.)

3 Kullanıcılar ve Gruplar

3.1 Ben kimim?

Linux iki tür kimlik tanır:

- Bir **kullanıcının** (user) bir oturum adı ve bir numarası, yani **UID**'si (user ID, kullanıcı kimliği) vardır. Çekirdek yalnızca numarayı saklar ve karşılaştırır. `root` kullanıcısının UID'si 0'dır ve o **süper kullanıcıdır** (superuser). Birçok *sistem kullanıcısı* (örneğin `daemon` ya da `www-data`) yalnızca hizmetleri sınırlı haklarla çalıştırmak için vardır.
- Bir **grup** (group), numarası (**GID**, group ID) olan, adlandırılmış bir kullanıcı kümesidir. Her kullanıcının

tam olarak bir *birincil grubu* (primary group) vardır ve birkaç *ek gruba* (supplementary groups) üye olabilir. Ubuntu her kullanıcı için aynı adlı özel bir grup oluşturur.

Her süreç, onu başlatan kullanıcının UID ve GID'lerini (*kimlik bilgilerini*, credentials) taşır; oluşturduğu bir dosyanın sahibi bu UID, grubu da birincil GID olur.

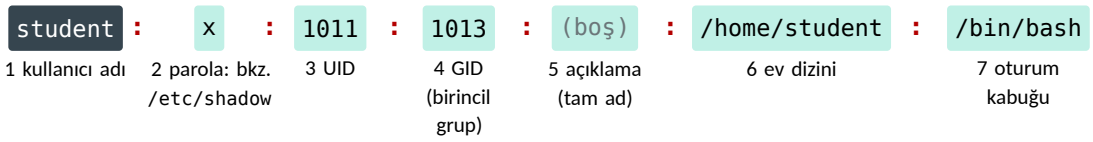
```
student@okan-lab:~/lab$ whoami
student
student@okan-lab:~/lab$ id
uid=1011(student) gid=1013(student) groups=1013(student)
student@okan-lab:~/lab$ groups
student
```

Burada `uid =1011(student)` kullanıcı numarası ve adı, `gid =1013(student)` birincil grup, `groups =` ise bütün gruplardır. Yeni kurulmuş bir Ubuntu'da ilk kullanıcının UID'si genellikle 1000'dir; laboratuvar makinemizde daha çok hesap vardır.

3.2 Hesap dosyaları `/etc/passwd` ve `/etc/group`

```
student@okan-lab:~/lab$ grep "^root:" /etc/passwd
root:x:0:0:root:/root:/bin/bash
student@okan-lab:~/lab$ grep "^$USER:" /etc/passwd
student:x:1011:1013::/home/student:/bin/bash
student@okan-lab:~/lab$ grep "^$USER:" /etc/group
student:x:1013:
student@okan-lab:~/lab$ getent group students
students:x:1010:
student@okan-lab:~/lab$ ls -l /etc/passwd /etc/group /etc/shadow
-rw-r--r-- 1 root root    809 Oct 12 11:50 /etc/group
-rw-r--r-- 1 root root   1598 Oct 12 11:50 /etc/passwd
-rw-r----- 1 root shadow  884 Oct 12 11:50 /etc/shadow
student@okan-lab:~/lab$ cat /etc/shadow
cat: /etc/shadow: Permission denied
```

`/etc/passwd` dosyasının bir satırında iki nokta üst üste ile ayrılmış yedi alan vardır:



Şekil 2: `/etc/passwd` satırının yedi alanı.

`/etc/group` satırında dört alan vardır: grup adı, parola (kullanılmaz, `x`), GID ve üyelerin virgülle ayrılmış listesi. `students` grubunun henüz üyesi yoktur (dördüncü alan boş).

Parola neden `/etc/passwd` içinde değil? Her program `/etc/passwd` dosyasını *okuyabilmelidir* — `ls`, UID'leri adlara çevirmek için ona ihtiyaç duyar. Bu yüzden parola *özetleri* (hash), modu `-rw-r-----` olan ve yalnızca `root` ile `shadow` grubunun okuyabildiği `/etc/shadow` dosyasında tutulur. `cat /etc/shadow` komutunun bizim için başarısız olmasının nedeni budur. Yöneticiler bu dosyaları asla elle düzenlemez; `useradd`, `usermod`, `groupadd` ve `passwd` komutlarını kullanırlar.

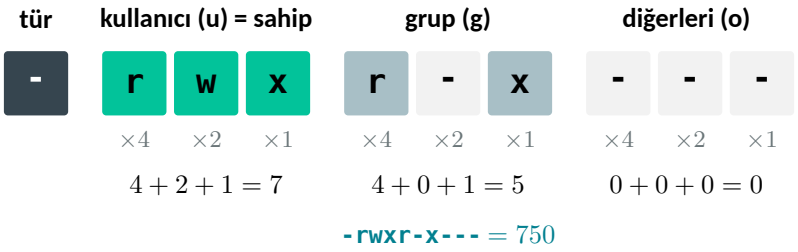
4 İzinleri Okumak

4.1 `ls -l` çıktısının sütunları

```
student@okan-lab:~/lab$ ls -l docs
total 24
-rw-rw-r-- 1 student student  53 Oct 12 19:38 'final report.txt'
-rw-rw-r-- 1 student student 302 Oct 12 19:38  linux_quotes.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_01.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_02.txt
-rw-rw-r-- 1 student student  16 Oct 12 19:38  notes_03.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38  syllabus.txt
student@okan-lab:~/lab$ ls -ld docs
drwxrwxr-x 2 student student 4096 Oct 12 19:38 docs
```

Sütunlar şunlardır: (1) dosya türü ve izinler, (2) bağlantı sayısı (7. ve 10. haftalar), (3) sahip, (4) grup, (5) bayt cinsinden boyut, (6) değişiklik zamanı ve (7) ad. Ad dışındaki her şey düğümden gelir; ad ise dizin girdisinden gelir. Düğüm sahip ve grup için *numaraları* saklar; `ls` adları `/etc/passwd` ve `/etc/group` içinde arayarak gösterir. `ls -l docs` (`docs` *içindeki* dosyalar) ile `ls -ld docs` (dizinin kendisi) arasındaki farka dikkat edin.

4.2 Mod dizgesini çözmek



Şekil 3: İlk karakter türdür; ardından kullanıcı, grup ve diğerleri için üç üçlü gelir; her bitin ağırlığı 4, 2 ya da 1'dir.

- **1. karakter, tür:** `-` normal dosya, `d` dizin, `l` sembolik bağlantı, `c`/`b` karakter/blok aygıtı.
- **2–10. karakterler:** her zaman kullanıcı (sahip), grup, diğerleri sırasıyla üç *üçlü* (triplet); her üçlünün içinde her zaman r, w, x. Harf iznin verildiğini, tire verilmediğini gösterir.

Her üçlüde 3 bit vardır, dolayısıyla $2^3 = 8$ olası üçlü vardır — tam olarak sekiz sekizli (octal) basamak, 0–7. $r = 4$, $w = 2$, $x = 1$ ağırlıklarıyla her üçlü tek bir basamaktır ve dokuz bit üç sekizli basamakla yazılır: `-rwxr-x---`, 750'dir. `stat` komutu iki biçimi de gösterir; `stat -c` ile alanları siz seçersiniz:

```
student@okan-lab:~/lab$ stat -c '%A  %a  %U  %G  %n' docs/syllabus.txt docs src/hello.c
-rw-rw-r--  664  student  student  docs/syllabus.txt
drwxrwxr-x  775  student  student  docs
-rw-rw-r--  664  student  student  src/hello.c
```

4.3 r, w ve x ne anlama gelir?

Bit	Normal bir dosyada	Bir dizinde
r (4)	içeriği okumak (`cat`, `less`, `cp`)	içindeki adları listelemek (`ls`)
w (2)	içeriği değiştirmek (düzenleyici, >>)	girdi oluşturmak, silmek ve yeniden adlandırmak (x ile birlikte)
x (1)	program ya da betik olarak çalıştırmak	içine girmek ve içindekilere ulaşmak (`cd`, içinden geçen yollar)

7. haftadan hatırlayın: dizin, (ad, düğüm numarası) çiftlerinden oluşan bir tablodur. Tabloyu okumak adları listelemek, tabloya yazmak girdi eklemek ya da kaldırmak demektir; x ("arama", search) ise tablo üzerinden bir yolu izlemektir. Bunun yeni başlayanların çoğunu şaşırtan iki sonucu vardır: bir dosyayı silmek *dizinde* yapılan bir değişikliktir, dolayısıyla dizinde w gerektirir ve dosyanın kendi w bitine bağlı değildir; bir dizinde x yoksa, dosyaların kendi izinleri ne kadar cömert olursa olsun, altındaki hiçbir şeye ulaşılamaz. Kısım 7 ikisini de gösteriyor.

4.4 Hangi sınıf uygulanır? Yalnızca biri

Bir süreç bir dosyaya eriştiğinde çekirdek üç üçlüyü birleştirmez; tam olarak birini seçer:



Şekil 4: Çekirdeğin erişim denetimi eşleşen ilk sınıfta durur.

Yani sahip, diğerleri üçlüsü daha cömert olsa bile *yalnızca* sahip üçlüsüne göre değerlendirilir. Gösterimimiz (`code/demo_order.sh`):

```
student@okan-lab:~/lab$ echo "hello" > me.txt
student@okan-lab:~/lab$ chmod 074 me.txt
student@okan-lab:~/lab$ ls -l me.txt
----rwxr-- 1 student student 6 Oct 12 19:38 me.txt
student@okan-lab:~/lab$ cat me.txt
cat: me.txt: Permission denied
student@okan-lab:~/lab$ chmod 644 me.txt
student@okan-lab:~/lab$ cat me.txt
hello
```

074 = `----rwxr--` modunda geri kalan herkes okuyabilir, ama sahip okuyamaz. Sahip durumu yine de düzeltebilir, çünkü modu değiştirmek r ya da w değil, *sahiplik* gerektirir. (`root` için okuma ve yazmaya her zaman izin verilir; çalıştırmaya ise en az bir x biti kuruluysa izin verilir.)

5 İzinleri chmod ile Değiştirmek

Bir dosyanın modunu yalnızca sahibi (ya da `root`) değiştirebilir. `chmod` (*change mode*, modu değiştir) iki gösterim kabul eder.

5.1 Sembolik mod

Biçim şöyledir: `chmod` *kim işleç izinler dosya…*:

- **kim:** u (user, kullanıcı = sahip), g (group, grup), o (others, diğerleri), a (all, üçü birden);
- **işleç:** + ekle, - kaldır, = tam olarak ata (o sınıfın adı verilmeyen bitleri temizlenir);
- **izinler:** `r`, `w`, `x` harflerinden herhangi biri.

Birden çok ifade *boşluksuz* virgülle ayrılır: `u =rw,g =r,o =` (boş `o =`, "diğerlerine hiçbir hak yok" demektir). Aşağıdaki oturum (`code/demo_chmod_sym.sh`) `docs/syllabus.txt` dosyasını adım adım değiştiriyor:

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u+x syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxrw-r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod go-w syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxr--r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod a=r syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-r--r--r-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u=rw,g=r,o= syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rw-r----- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod u+rwx,g+w syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxrw---- 1 student student 578 Oct 12 19:38 syllabus.txt
```

`chmod` başarılı olduğunda hiçbir şey yazdırmaz ("haber olmaması iyi haberdir"); her zaman `ls -l` ya da `stat` ile denetleyin. + ve - öbür bitlere dokunmaz; bu da sembolik modu "grup da yazabilsin" (`chmod g+w`) gibi küçük değişiklikler için ideal kılar. Sınıf belirtmeden `chmod +x file` yazmak, x'i *umask'ın engellemediği* her sınıfa ekler (Kısım 6); u+x daha açıktır.

5.2 Sekizli mod

Sekizli modda dokuz bitin hepsini birden, sınıf başına bir basamakla verirsiniz:

Sekizli	İkili $r\,w\,x$	Sembolik	Anlamı
0	000	`---`	erişim yok
1	001	`--x`	yalnızca çalıştırma
2	010	`-w-`	yalnızca yazma (nadir)
3	011	`-wx`	yazma ve çalıştırma (nadir)
4	100	`r--`	yalnızca okuma
5	101	`r-x`	okuma ve çalıştırma
6	110	`rw-`	okuma ve yazma
7	111	`rwx`	her şey

4, 2 ve 1 ikinin kuvvetleri olduğundan 0'dan 7'ye kadar her basamak bunların *tek bir biçimde* toplamıdır: bu, üçer bitlik ikili sayıdan başka bir şey değildir. İki hızlı denetim: tek bir basamak her zaman x içerir; 4 ya da daha büyük bir basamak her zaman r içerir.

Çözümlü örnek: iki yönde dönüştürmek

Sembolik → sekizli. `rwxr-xr--`: `rwx` $= 4+2+1 = 7$, `r-x` $= 4+0+1 = 5$, `r--` $= 4+0+0 = 4$; mod 754'tür. Benzer biçimde `rw-rw-r--` $= 664$ ve `rw-r-----` $= 640$.
Sekizli → sembolik. 711: $7 = 4{+}2{+}1 \to$ `rwx`, $1 \to$ `--x`, $1 \to$ `--x`; yani `rwx--x--x`. 640: $6 = 4{+}2 \to$ `rw-`, $4 \to$ `r--`, $0 \to$ `---`: `rw-r-----`.
(Bu notlardaki bütün dönüşümler `code/perm_check.py` ile denetlendi.)

Sekizli	Sembolik	Tipik kullanım
644	`rw-r--r--`	sıradan dosyalar: herkes okur, yalnızca sahip yazar
755	`rwxr-xr-x`	başkalarının da kullanabileceği programlar, betikler ve dizinler
600	`rw-------`	özel dosyalar (parolalar, SSH anahtarları — `ssh` başkalarının okuyabildiği bir anahtarı reddeder)
700	`rwx------`	özel dizinler (`~/.ssh`)
640 / 750	`rw-r-----` / `rwxr-x---`	grupla paylaşılan, diğerlerinden gizlenen
777	`rwxrwxrwx`	herkes değiştirebilir ya da silebilir: neredeyse hiçbir zaman doğru değildir

Güvenlik: bir hatayı asla `chmod 777` ile "düzeltmeyin"

`chmod 777` (ya da daha kötüsü `chmod -R 777`), makinedeki *her* kullanıcının ve ele geçirilmiş her programın dosyayı değiştirmesine ya da yerine başkasını koymasına izin vererek bir Permission-denied iletisini ortadan kaldırır. Eksik olan tek biti bulun ve yalnızca onu ekleyin, örneğin `chmod u+x script.sh`. Bu, *en az ayrıcalık ilkesidir* (principle of least privilege).

Sekizli oturum (`code/demo_chmod_oct.sh`) iki tipik hatayı da gösteriyor:

```
student@okan-lab:~/lab$ cd docs
student@okan-lab:~/lab/docs$ chmod 754 syllabus.txt
student@okan-lab:~/lab/docs$ ls -l syllabus.txt
-rwxr-xr-- 1 student student 578 Oct 12 19:38 syllabus.txt
student@okan-lab:~/lab/docs$ chmod 640 syllabus.txt
student@okan-lab:~/lab/docs$ chmod 600 notes_01.txt
student@okan-lab:~/lab/docs$ chmod 644 notes_02.txt
student@okan-lab:~/lab/docs$ chmod 700 ../backup
student@okan-lab:~/lab/docs$ stat -c '%a  %A  %n' syllabus.txt notes_01.txt notes_02.txt ../
    backup
```

```
640  -rw-r-----  syllabus.txt
600  -rw-------  notes_01.txt
644  -rw-r--r--  notes_02.txt
700  drwx------  ../backup
student@okan-lab:~/lab/docs$ chmod 854 notes_03.txt
chmod: invalid mode: ‘’854
Try 'chmod --help' for more information.
student@okan-lab:~/lab/docs$ chmod rw notes_03.txt
chmod: invalid mode: ‘’rw
Try 'chmod --help' for more information.
```

Sekizli sistemde 8 ve 9 basamakları yoktur; `rw` ise ne bir sayıdır ne de geçerli bir sembolik ifade (*kim* ve işleç eksik: u+rw). Özyinelemeli `chmod -R` aynı modu dosyalara *ve* dizinlere uygular; `chmod -R 644 dir` her alt dizinden x'i kaldırır ve sizi dışarıda bırakır. `-R` ile yalnızca hak kaldıran sembolik modu (`chmod -R go-rwx dir`) ya da x'i yalnızca dizinlere (ve zaten x'i olan dosyalara) ekleyen büyük X'i tercih edin.

5.3 Bir betiği çalıştırılabilir yapmak

Yeni oluşturulan bir dosya asla çalıştırılabilir değildir (nedenini Kısım 6 açıklıyor). `code/demo_script.sh` oturumu:

```
student@okan-lab:~/lab$ cd scripts
student@okan-lab:~/lab/scripts$ echo '#!/bin/bash' > hello.sh
student@okan-lab:~/lab/scripts$ echo 'echo "Hello, $USER! You are in $PWD"' >> hello.sh
student@okan-lab:~/lab/scripts$ cat hello.sh
#!/bin/bash
echo "Hello, $USER! You are in $PWD"
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rw-rw-r-- 1 student student 49 Oct 12 19:38 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
bash: ./hello.sh: Permission denied
student@okan-lab:~/lab/scripts$ bash hello.sh
Hello, student! You are in /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ chmod u+x hello.sh
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rwxrw-r-- 1 student student 49 Oct 12 19:38 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
Hello, student! You are in /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ hello.sh
bash: hello.sh: command not found
student@okan-lab:~/lab/scripts$ cd ../src
student@okan-lab:~/lab/src$ gcc -Wall -O2 hello.c -o hello
student@okan-lab:~/lab/src$ ls -l hello.c hello
-rwxrwxr-x 1 student student 15960 Oct 12 19:38 hello
-rw-rw-r-- 1 student student    90 Oct 12 19:38 hello.c
student@okan-lab:~/lab/src$ ./hello
Hello from the lab!
```

- `./hello.sh` yazmak, çekirdekten dosyayı *çalıştırmasını* ister (`execve` sistem çağrısı); bunun için x gerekir. `bash hello.sh` ise `bash`'i çalıştırır ve o da dosyayı yalnızca *okur* — r yeterlidir.
- `./` olmadan `hello.sh` "command not found" (komut bulunamadı) verir: kabuk komutları yalnızca PATH içinde listelenen dizinlerde arar ve çalışma dizini (güvenlik nedeniyle) bunlar arasında değildir. Bu bir izin sorunu değildir.

- İlk satır `#!/bin/bash` (*shebang*), betiği hangi yorumlayıcının çalıştıracağını çekirdeğe söyler. 14. hafta bunun üzerine kurulacak.
- `gcc` x bitlerini kendisi koyar: programı 777 ile oluşturur, umask 002 bunu 775'e indirir.

6 Varsayılan İzinler: `umask`

Yeni dosyalar neden 664, yeni dizinler neden 775? Bir dosya oluşturan program, `open()` ya da `mkdir()` çağrısında bir mod ister: düzenleyiciler, `touch` ve kabuk dosyaları için 666 (`rw-rw-rw-`), dizinler için 777 ister. Çekirdek ardından sürecin **umask** değerinde (user file-creation mask, kullanıcı dosya oluşturma maskesi) kurulu olan her biti *kaldırır*. umask, çocuk süreçlere aktarılan, süreç başına bir ayardır; oturum açılırken ayarlanır (Ubuntu'da normal kullanıcılar için 002, root için 022) ve kabuğun yerleşik `umask` komutuyla değiştirilebilir.

6.1 Bitleri temizler, çıkarma yapmaz

Biçimsel olarak, bit bit uygulanan

$$\text{yeni mod} = \text{istenen mod} \ \ \text{AND} \ \ (\text{NOT umask})$$

işlemidir. Alışılmış 022 umask'ı için bu bir çıkarma gibi görünür, $666 - 022 = 644$:

istenen	$666 =$	$110\ 110\ 110$
umask	$022 =$	$000\ 010\ 010$
NOT umask		$111\ 101\ 101$
AND		$110\ 100\ 100 = 644 =$ `rw-r--r--`

Çözümlü örnek: çıkarmanın yanıldığı yer, umask 033

033 umask'ı şu demektir: gruptan ve diğerlerinden w ve x'i al. Bir *dosya* için:

istenen	$666 =$	$110\ 110\ 110$
NOT 033		$111\ 100\ 100$
AND		$110\ 100\ 100 = 644 =$ `rw-r--r--`

Düz çıkarma $666 - 033 = 633 =$ `rw--wx-wx` verirdi: okuma olmadan yazma ve çalıştırma, üstelik hiç istenmemiş x bitleriyle. Anlamsız! Her basamakta $6 - 3 = 3$ hiç kurulmamış bir x bitini "ödünç alır"; bit işlemi ise olmayan biti olmadığı gibi bırakır: 6 AND NOT $3 = 110$ AND $100 = 100 = 4$.
Bir *dizin* için 777'nin her biti kuruludur, dolayısıyla çıkarma tesadüfen aynı sonucu verir: $777 - 033 = 744$. Genel olarak çıkarma, umask'ın her biti istenen modda da kuruluysa ve yalnızca o zaman doğrudur. Dosyalarda (666, x biti yok) bu yüzden x biti içeren her umask için yanılır; örneğin 027: $666 - 027 = 637$ (yanlış), doğrusu 640; ve 077: $666 - 077 = 567$ (yanlış), doğrusu 600.

umask	**yeni dosya** (666)	**yeni dizin** (777)	**tipik kullanım**
002	664 `rw-rw-r--`	775 `rwxrwxr-x`	Ubuntu kullanıcıları (özel gruplar)
022	644 `rw-r--r--`	755 `rwxr-xr-x`	root, birçok sunucu
027	640 `rw-r-----`	750 `rwxr-x---`	grup okur, diğerleri hiçbir şey
077	600 `rw-------`	700 `rwx------`	tamamen özel
033	644 `rw-r--r--`	744 `rwxr--r--`	çıkarma tuzağı

Gerçek oturum (`code/demo_umask.sh`) her satırı doğruluyor:

```
student@okan-lab:~/lab$ mkdir umask_test
student@okan-lab:~/lab$ cd umask_test
student@okan-lab:~/lab/umask_test$ umask
0002
student@okan-lab:~/lab/umask_test$ umask -S
u=rwx,g=rwx,o=rx
student@okan-lab:~/lab/umask_test$ touch f_002
student@okan-lab:~/lab/umask_test$ mkdir d_002
student@okan-lab:~/lab/umask_test$ umask 022
student@okan-lab:~/lab/umask_test$ touch f_022
student@okan-lab:~/lab/umask_test$ mkdir d_022
student@okan-lab:~/lab/umask_test$ umask 077
student@okan-lab:~/lab/umask_test$ touch f_077
student@okan-lab:~/lab/umask_test$ mkdir d_077
student@okan-lab:~/lab/umask_test$ umask 033
student@okan-lab:~/lab/umask_test$ touch f_033
student@okan-lab:~/lab/umask_test$ mkdir d_033
student@okan-lab:~/lab/umask_test$ umask 002
student@okan-lab:~/lab/umask_test$ stat -c '%a  %A  %n' f_* d_*
664  -rw-rw-r--  f_002
644  -rw-r--r--  f_022
644  -rw-r--r--  f_033
600  -rw-------  f_077
775  drwxrwxr-x  d_002
755  drwxr-xr-x  d_022
744  drwxr--r--  d_033
700  drwx------  d_077
```

`umask -S` tümleyeni sembolik biçimde, yani *izin verileni* yazdırır. Dosyalar x olmadan istendiği için hiçbir umask yeni bir dosyayı çalıştırılabilir yapamaz: bunun için her zaman bir `chmod` gerekir. umask kabuk sürecinin bir özelliğidir; `umask` komutunun (tıpkı `cd` gibi) kabuğun içine yerleşik olmasının nedeni budur. Bir değişiklik kabuktan çıkılana kadar sürer; kalıcı yapmak için örneğin `umask 027` satırını `~/.bashrc` dosyasına ekleyin. Var olan dosyalar hiçbir zaman etkilenmez.

7 Uygulamada Dizin İzinleri

`code/demo_dirperm.sh` oturumu içinde tek bir dosya olan `box` adlı bir dizin oluşturur ve ondan bitler kaldırır:

```
student@okan-lab:~/lab$ mkdir box
student@okan-lab:~/lab$ echo "top secret" > box/secret.txt
student@okan-lab:~/lab$ ls -ld box
drwxrwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ chmod u-x box
student@okan-lab:~/lab$ ls -ld box
drw-rwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ ls box
secret.txt
student@okan-lab:~/lab$ ls -l box
ls: cannot access 'box/secret.txt': Permission denied
total 0
-????????? ? ? ? ?            ? secret.txt
```

```
student@okan-lab:~/lab$ cd box
bash: cd: box: Permission denied
student@okan-lab:~/lab$ cat box/secret.txt
cat: box/secret.txt: Permission denied
student@okan-lab:~/lab$ chmod u-r,u+x box
student@okan-lab:~/lab$ ls -ld box
d-wxrwxr-x 2 student student 4096 Oct 12 19:38 box
student@okan-lab:~/lab$ ls box
ls: cannot open directory 'box': Permission denied
student@okan-lab:~/lab$ cat box/secret.txt
top secret
```

- r **var,** x **yok:** `ls` ad listesini okuyabilir, ama `ls -l` düğümlere dizin üzerinden bakamaz — soru işaretlerinin nedeni budur — ve `cd` ile `cat` başarısız olur.
- x **var,** r **yok:** dizin listelenemez, ama adını bildiğiniz bir dosya açılabilir. (Her odanın anahtarı elinizde olan ama odaların listesi olmayan bir bina gibi.)

Uygulamada dizinlerde r ve x birlikte bulunur: `r-x` ya da `rwx`.

```
student@okan-lab:~/lab$ chmod 755 box
student@okan-lab:~/lab$ chmod 444 box/secret.txt
student@okan-lab:~/lab$ ls -l box
total 4
-r--r--r-- 1 student student 11 Oct 12 19:38 secret.txt
student@okan-lab:~/lab$ echo "more" >> box/secret.txt
bash: box/secret.txt: Permission denied
student@okan-lab:~/lab$ rm box/secret.txt
rm: remove write-protected regular file 'box/secret.txt'? y
student@okan-lab:~/lab$ ls box
student@okan-lab:~/lab$ chmod 555 box
student@okan-lab:~/lab$ touch box/new.txt
touch: cannot touch 'box/new.txt': Permission denied
student@okan-lab:~/lab$ chmod 755 box
```

Dosya salt okunurdur (444), bu yüzden sonuna ekleme başarısız olur — ama `rm` yalnızca bir soru sorar ve dosyayı siler, çünkü silmek yazılabilir olan *dizini* (`box`) değiştirir. Ancak dizinin kendisi 555 olduğunda hiçbir şey eklenemez ya da kaldırılamaz.

> **Güvenlik: `rm` ve salt okunur dosyalar**
>
> Salt okunur bir dosya silinmeye karşı korunmuş *değildir*. `rm -f` soruyu bile atlar; `rm -r` ise bütün ağaçları geri dönüşüm kutusu olmadan siler (10. hafta). Dosyaları silinmeye karşı korumak için onları içeren dizinden w'yi kaldırın.

8 Sahiplik: `chown` ve `chgrp`

`chown user file` sahibi (`chown user:group file` ikisini birden), `chgrp group file` grubu değiştirir; ikisi de `-R` kabul eder. Linux'taki kurallar:

- Sahibi yalnızca `root` değiştirebilir. Aksi hâlde bir kullanıcı disk kotasından kaçmak için dosyalarını başkasına "devredebilir" ya da bir dosyayı başkasınınmış gibi gösterebilirdi.

- Sahip grubu değiştirebilir, ama yalnızca üyesi olduğu bir gruba.

```
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student student 16 Oct 12 19:38 docs/notes_01.txt
student@okan-lab:~/lab$ chown root docs/notes_01.txt
chown: changing ownership of 'docs/notes_01.txt': Operation not permitted
student@okan-lab:~/lab$ chgrp students docs/notes_01.txt
chgrp: changing group of 'docs/notes_01.txt': Operation not permitted
student@okan-lab:~/lab$ chown $USER docs/notes_01.txt
```

İletinin farklı olduğuna dikkat edin: *Operation not permitted* (işleme izin verilmedi, EPERM), izin bitleri ne olursa olsun işlemin bir ayrıcalık gerektirdiği anlamına gelir; *Permission denied* (EACCES) ise bir izin bitinin eksik olduğu anlamına gelir.

8.1 Bir klasörü bir grupla paylaşmak

Bir sonraki oturum için yönetici `sudo usermod -aG students student` komutunu çalıştırdı (`students` ek grubuna, `-G`, ekle, `-a`) ve öğrenci yeniden oturum açtı — grup üyeliği oturum açılırken okunur. (`-a` olmadan `usermod -G`, kullanıcının ek gruplarını *değiştirir*: klasik bir yönetici hatası.)

```
student@okan-lab:~/lab$ id
uid=1011(student) gid=1013(student) groups=1013(student),1010(students)
student@okan-lab:~/lab$ groups
student students
student@okan-lab:~/lab$ chgrp students docs/notes_01.txt
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-rw-r-- 1 student students 16 Oct 12 19:38 docs/notes_01.txt
student@okan-lab:~/lab$ mkdir shared
student@okan-lab:~/lab$ chgrp students shared
student@okan-lab:~/lab$ chmod 2770 shared
student@okan-lab:~/lab$ ls -ld shared
drwxrws--- 2 student students 4096 Oct 12 19:38 shared
student@okan-lab:~/lab$ touch shared/plan.txt
student@okan-lab:~/lab$ ls -l shared
total 0
-rw-rw-r-- 1 student students 0 Oct 12 19:38 plan.txt
```

Artık `chgrp` çalışıyor. 2770 modunun başta dördüncü bir basamağı vardır: 2, grubun x konumunda `s` olarak görünen **setgid** bitidir. Bir dizinde, yeni dosyaların dizinin grubunu devralmasını sağlar: `plan.txt`, oluşturanın birincil grubuna değil, `students` grubuna aittir. Bu yüzden bir ekip klasörü üç adım ister: ortak bir grup, klasörü `chgrp` ile bu gruba vermek ve `chmod 2770`.

9 root ve sudo

`root` (UID 0) okuma ve yazma denetimlerine tabi değildir; sahipleri değiştirebilir, yazılım kurabilir ve sistemi yeniden yapılandırabilir. Sürekli root olarak çalışmak tehlikelidir: tek bir yazım hatası sistemi yok edebilir. Bu yüzden Ubuntu'da varsayılan olarak root parolası yoktur; yöneticiler bunun yerine tek tek komutları `sudo` (*substitute user do*) ile çalıştırır:

- `sudo komut`, *tek* bir komutu root olarak çalıştırır;
- root'un değil, *sizin* parolanızı sorar ve onu terminal başına yaklaşık 15 dakika hatırlar (`sudo -k`

unutturur);

- yalnızca `/etc/sudoers` dosyasında izin verilen kullanıcılar kullanabilir — Ubuntu'da `sudo` grubunun üyeleri;
- her kullanım günlüğe kaydedilir.

Yönetici yapılmadan önce öğrencimiz (parola yazılır ama gösterilmez):

```
student@okan-lab:~/lab$ sudo whoami
[sudo] password for student:
student is not in the sudoers file.
```

Yönetici öğrenciyi `sudo` grubuna ekledikten sonra (root'un yaptığı bir hazırlık adımı):

```
student@okan-lab:~/lab$ groups
student sudo students
student@okan-lab:~/lab$ whoami
student
student@okan-lab:~/lab$ sudo whoami
[sudo] password for student:
root
student@okan-lab:~/lab$ ls -l docs/notes_02.txt
-rw-rw-r-- 1 student student 16 Oct 12 19:38 docs/notes_02.txt
student@okan-lab:~/lab$ chown root docs/notes_02.txt
chown: changing ownership of 'docs/notes_02.txt': Operation not permitted
student@okan-lab:~/lab$ sudo chown root:root docs/notes_02.txt
student@okan-lab:~/lab$ ls -l docs/notes_02.txt
-rw-rw-r-- 1 root root 16 Oct 12 19:38 docs/notes_02.txt
student@okan-lab:~/lab$ echo "more" >> docs/notes_02.txt
bash: docs/notes_02.txt: Permission denied
student@okan-lab:~/lab$ sudo -k
```

Bu notlarda bir komutun root ayrıcalıklarıyla çalıştığı tek çıktı budur ve öyle olduğu belirtilmiştir. `sudo chown root:root` sonrasında mod hâlâ 664'tür, ama `student` artık yalnızca r hakkı olan *diğerleri* sınıfına düşer: sonuna ekleme başarısız olur.

> **Güvenlik: sudo**
>
> Her `sudo` komutunu Enter'a basmadan önce iki kez okuyun. Root olarak `sudo rm -r / home/x` (`/` işaretinden sonraki boşluğa dikkat) bütün sistemi silmeye çalışır. Anlamadığınız `sudo` komutlarını internetten asla yapıştırmayın ve `sudo`'yu asla kendi dosyalarınızdaki bir Permission-denied iletisinden kurtulmak için kullanmayın — bunun yerine izinleri düzeltin.

10 Özel Bitler: setuid ve Yapışkan Bit

Modda dokuz izin bitinin yanında, başta dördüncü bir sekizli basamak olarak yazılan üç özel bit daha vardır: setuid $= 4000$, setgid $= 2000$ (yukarıda gördük), yapışkan bit (sticky bit) $= 1000$. Toplamda bu, $2^{12} = 4096$ olası mod eder.

10.1 setuid: `passwd` `/etc/shadow`'a nasıl yazabilir?

```
student@okan-lab:~/lab$ ls -l /usr/bin/passwd /etc/shadow
-rw-r----- 1 root shadow   884 Oct 12 11:50 /etc/shadow
-rwsr-xr-x 1 root root   64152 May 30  2024 /usr/bin/passwd
student@okan-lab:~/lab$ ls -l /usr/bin/sudo /usr/bin/su
-rwsr-xr-x 1 root root  55680 Mar  6  2026 /usr/bin/su
-rwsr-xr-x 1 root root 277936 Mar  2  2026 /usr/bin/sudo
```

Kullanıcılar kendi parolalarını değiştirebilmelidir, ama `/etc/shadow` dosyasına yalnızca root yazabilir. Bu yüzden `/usr/bin/passwd` programının sahibi root'tur ve **setuid** biti kuruludur (sahibin x konumundaki `s`, 4755). setuid bir dosyadan başlatılan süreç, dosya sahibinin — burada root'un — *etkin* (effective) UID'siyle çalışır; *gerçek* (real) UID'si ise `student` olarak kalır, böylece `passwd` kimin parolasını değiştireceğini bilir. Bu, küçük bir güvenilir kod parçasına verilen ayrıcalıktır; 2. haftadaki sistem çağrısıyla aynı fikirdir. setuid-root bir programdaki bir hata saldırgana root yetkisi verir; bu yüzden bu programlar çok azdır (`passwd`, `su`, `sudo`, …) ve kendi betiklerinizi asla setuid yapmamalısınız (Linux betiklerde setuid'i zaten yok sayar).

10.2 `/tmp` üzerindeki yapışkan bit

```
student@okan-lab:~/lab$ ls -ld /tmp
drwxrwxrwt 16 root root 4096 Oct 12 19:38 /tmp
student@okan-lab:~/lab$ stat -c '%a  %A  %n' /usr/bin/passwd /tmp
4755  -rwsr-xr-x  /usr/bin/passwd
1777  drwxrwxrwt  /tmp
student@okan-lab:~/lab$ ls -l /tmp/w11_admin.txt
-rw-rw-rw- 1 root root 12 Oct 12 19:38 /tmp/w11_admin.txt
student@okan-lab:~/lab$ rm /tmp/w11_admin.txt
rm: cannot remove '/tmp/w11_admin.txt': Operation not permitted
student@okan-lab:~/lab$ echo "my notes" > /tmp/w11_mine.txt
student@okan-lab:~/lab$ ls -l /tmp/w11_mine.txt
-rw-rw-r-- 1 student student 9 Oct 12 19:38 /tmp/w11_mine.txt
student@okan-lab:~/lab$ rm /tmp/w11_mine.txt
```

`/tmp` dizininin modu 1777 = `rwxrwxrwt`'dir: herkes orada dosya oluşturabilir. Başka bir kural olmasaydı, dizindeki w herkesin başkalarının dosyalarını silmesine izin verirdi (Kısım 7). **Yapışkan bit** (sticky bit; diğerlerinin x konumundaki `t`) şu kuralı ekler: böyle bir dizinde bir dosyayı yalnızca dosyanın (ya da dizinin) sahibi veya root silebilir ya da yeniden adlandırabilir. Bu gösterim için root herkese yazılabilir bir `/tmp/w11_admin.txt` dosyası oluşturmuştu; öğrencimiz onu yine de silemez, ama kendi dosyasını silebilir.

11 Komut Başvuru Tablosu

Komut	Amaç	Örnek
`whoami`, `id`, `groups`	kullanıcı adı; UID, GID, gruplar	`id`
`getent group G`	grup veritabanından bir satır	`getent group students`
`ls -l`, `ls -ld`	mod, sahip, grup (`-d` ile dizinin kendisininki)	`ls -ld docs`
`stat -c FORMAT`	%a sekizli, %A sembolik, %U/%G sahip/grup, %n ad	`stat -c '%a %A %n' *`
`chmod who± =perm`	bitleri sembolik olarak değiştir	`chmod u+x,go-w f`
`chmod NNN`	dokuz bitin hepsini sekizli olarak ata	`chmod 640 f`
`chmod -R`	özyinelemeli (sembolik tercih edin)	`chmod -R go-rwx dir`
`umask`, `umask -S`	oluşturma maskesini göster ya da ayarla	`umask 027`
`chown U[:G]`	sahibi değiştir (yalnızca root)	`sudo chown root:root f`
`chgrp G`	grubu değiştir (gruplarınızdan birine)	`chgrp students dir`
`sudo CMD`	tek bir komutu root olarak çalıştır	`sudo whoami`
`sudo -k`	önbellekteki sudo parolasını unut	`sudo -k`
`usermod -aG G U`	bir kullanıcıyı bir gruba ekle (yönetici)	`sudo usermod -aG students ali`

Ayrıntılar man sayfalarındadır (`man chmod`; NAME, SYNOPSIS, DESCRIPTION bölümleri; `/sözcük` arar, `q` çıkar); `chmod --help` kısa bir özet yazdırır.

12 Sık Görülen Hatalar ve Anlamları

İleti	Nedeni ve çözümü
`bash: ./hello.sh: Permission denied`	Dosyanın x biti yok. `chmod u+x hello.sh` (ya da `bash hello.sh` ile çalıştırın).
`bash: hello.sh: command not found`	İzin sorunu değil: `.` PATH içinde değil. `./hello.sh` yazın.
`cat: /etc/shadow: Permission denied`	Diğerleri sınıfındasınız ve bu sınıfın r'si yok. Sistem dosyaları için doğru davranış.
`bash: cd: box: Permission denied`	Dizinde sizin için x yok. `chmod u+x box`.
`ls: cannot open directory 'box': Permission denied`	Dizinde r yok. `chmod u+r box`.
`-????????? ? ? ? ? ? secret.txt`	r'si olup x'i olmayan bir dizinde `ls -l`: adlar görünür, düğümler görünmez.
`chown: …: Operation not permitted`	Sahibi yalnızca root değiştirebilir; yöneticiye başvurun (`sudo chown`).
`chgrp: …: Operation not permitted`	O grubun üyesi değilsiniz (ya da dosyanın sahibi değilsiniz).
`chmod: invalid mode: '854'`	8 ve 9 sekizli basamak değildir; ya da *kim*/işleç içermeyen bir sembolik mod (`rw`).
`rm: remove write-protected regular file?`	Dosya salt okunur, ama dizin yazılabilir: `y` yanıtı dosyayı siler.
`rm: cannot remove '/tmp/…': Operation not permitted`	Yapışkan bit: dosya sizin değil.
`student is not in the sudoers file.`	Yönetici değilsiniz; `sudo` reddedilir (ve günlüğe kaydedilir).

13 Özet

Sekiz cümlede 11. hafta

1. Her dosya erişimi bir sistem çağrısıdır; bu yüzden çekirdek (2. hafta) sürecin UID ve GID'lerini düğümde saklanan sahip, grup ve modla (7. hafta) karşılaştırabilir.
2. Kullanıcıların ve grupların adları ve numaraları vardır; `id` bunları gösterir, `/etc/passwd` ve `/etc/group` saklar, `/etc/shadow` parola özetlerini tutar.
3. Mod dizgesi bir tür karakteri ile r, w, x'ten oluşan üç üçlüdür (kullanıcı, grup, diğerleri); 4, 2, 1 ağırlıklarıyla her üçlü bir sekizli basamaktır, örneğin `-rwxr-x---` $= 750$.
4. Tam olarak bir sınıf uygulanır: sahip, değilse grup, değilse diğerleri.
5. Dizinlerde r listeler, w girdi oluşturur ve siler, x içeri girer; bir dosyayı silmek dosyada değil, dizinde w gerektirir.
6. `chmod` sembolik (`u+x`, `go-w`, `a =r`) ya da sekizli (`644`, `755`, `600`) çalışır; asla `777`.
7. umask $666/777$'den bit temizler: $002 \to 664/775$, $022 \to 644/755$; dosyalar için $033 \to 644$ (bit işlemi, 633 değil).
8. Sahipleri yalnızca root değiştirir; `sudo` tek bir komutu root olarak çalıştırır; setuid (`passwd`) ve yapışkan bit (`/tmp`) özel bitlerdir.

14 Sözlük

EACCES / EPERM "Permission denied" (bir bit eksik) ve "Operation not permitted" (bir ayrıcalık eksik) iletilerinin arkasındaki hata kodları.

GID (group ID) Bir grubun numarası; her kullanıcının bir birincil ve olası ek grupları vardır.

Mod (mode) Bir dosyanın türü ve düğümde saklanan 12 izin biti (dokuz rwx biti ile setuid, setgid ve yapışkan bit).

root Süper kullanıcı, UID 0.

Sekizli gösterim (octal notation) Her üçlüyü $r = 4$, $w = 2$, $x = 1$ ile 0–7 arasında tek bir basamak olarak yazmak.

setgid Bir dizinde: yeni dosyalar dizinin grubunu devralır (2000).

setuid Bir program, sahibinin etkin UID'siyle çalışır (4000); örneğin `passwd`.

sudo `/etc/sudoers` dosyasında izin verilen kullanıcılar için tek bir komutu root olarak çalıştırır.

UID (user ID) Çekirdeğin bir kullanıcıyı tanıdığı numara.

umask Yeni oluşturulan her dosya ya da dizinin modundan temizlenen bitlerin, süreç başına tutulan maskesi.

Yapışkan bit (sticky bit) Paylaşılan bir dizinde bir dosyayı yalnızca sahibi silebilir ya da yeniden adlandırabilir (1000); örneğin `/tmp`.

15 Laboratuvar Föyü: Bir Proje Klasörünü Güvenli Hâle Getirmek

`bash mcmp117_lab_setup.sh` komutunu çalıştırdıktan sonra `~/lab` içinde çalışın. Her alıştırma için kullandığınız komut(lar)ı *ve* sonucu kanıtlayan `ls -l` ya da `stat` satırını teslim edin. Toplam: 100 puan (A bölümü 20, B bölümü 48, C bölümü 32). Örnek çözümler için bkz. Kısım 18. Ek alıştırma: `ls -l /usr/bin | grep rws` makinenizdeki setuid programlarını listeler; `python3 code/perm_check.py` ise yanıtlarınızı denetlemeniz için bir dönüşüm tablosu yazdırır.

A bölümü --- Ben kimim? (temel)

1. **(4 puan)** `whoami`, `id` ve `groups` komutlarını çalıştırın. UID'nizi, birincil grubunuzu ve üyesi olduğunuz grup sayısını yazın.
2. **(5 puan)** `/etc/passwd` içindeki kendi satırınızı gösterin (`grep "^$USER:" /etc/passwd`). Yedi alanın hepsini adlandırın. Ev dizininiz ve oturum kabuğunuz nedir?
3. **(5 puan)** `ls -l docs/syllabus.txt` komutunu çalıştırın. Dosyayı kim okuyabilir, kim yazabilir? Modunu sekizli olarak verin ve `stat -c '%a %A'` ile denetleyin.
4. **(6 puan)** Kâğıt üzerinde: `rwxr-x---`, `rw-r--r--` ve `rwx------` modlarını sekizliye; 640, 711, 775'i sembolik gösterime çevirin.

B bölümü --- chmod ve umask (orta)

5. **(6 puan)** `scripts/hello.sh` betiğini, Kısım 5.3 içindeki iki `echo` komutuyla bu dosyaya yazarak oluşturun. Onu *yalnızca sahip* için çalıştırılabilir yapın ve `./scripts/hello.sh` olarak çalıştırın.
6. **(6 puan)** Yalnızca *sembolik* mod kullanarak `docs/notes_01.txt` dosyasını `rw-rw-r--`'dan `rw-r-----`'ya değiştirin.
7. **(6 puan)** Yalnızca *sekizli* mod kullanarak `docs/notes_02.txt` dosyasını sahibi tarafından okunabilir ve yazılabilir, başka hiç kimse tarafından erişilemez yapın. `stat` ile doğrulayın.
8. **(6 puan)** `projects/alpha` dizinini özel yapın: yalnızca siz listeleyebilir, girebilir ya da değiştirebilirsiniz. İçindeki dosyaları da değiştirmeniz gerekir mi? Neden (gerekmez)?
9. **(6 puan)** `projects/beta`: grup onu listeleyip içine girebilsin ama değiştiremesin; diğerleri hiçbir şey alamasın. `projects/beta/README.md` dosyasına `rw-r-----` modunu verin.
10. **(6 puan)** `umask 027` ile yeni bir dosyanın ve yeni bir dizinin modlarını tahmin edin. Doğrulayın, sonra `umask 002`'ye geri dönün.
11. **(6 puan)** Aynısını `umask 033` ile yapın. Dosyanın neden 633 olmadığını 666 ve 033'ün ikili gösterimiyle açıklayın.
12. **(6 puan)** İçinde `a.txt` dosyası bulunan `t12` adlı bir dizin oluşturun. `t12`'den sahibin x hakkını kaldırın. `ls t12`, `cd t12`, `cat t12/a.txt` komutlarından hangileri hâlâ çalışır? Açıklayın, sonra biti geri koyun.

C bölümü --- Zorlu

13. **(7 puan)** `docs/notes_03.txt` dosyasını salt okunur (444) yapın ve silin. Bu neden işe yarıyor? Sonra `docs/linux_quotes.txt` dosyasını salt okunur yapın, `docs`'u 555 yapın ve `rm -f` ile yeniden

deneyin. Farkı açıklayın ve `docs`'u 775'e geri getirin.

14. **(7 puan)** `src/hello.c` dosyasını `chown` ile `root`'a, `chgrp` ile `students` grubuna vermeyi deneyin. İki hata iletisini de açıklayın. Bir yönetici hangi komutu kullanırdı?

15. **(8 puan)** `/usr/bin/passwd`, `/etc/shadow` ve `/tmp`'nin modlarını gösterin. `s`'yi ve `t`'yi açıklayın. `/tmp` içinde bir dosya oluşturun, listeleyin ve silin.

16. **(10 puan) Ekip klasörü.** İçinde `deploy.sh` (`deploying...` yazdıran iki satırlık bir betik), `secret.txt` ve `notes.txt` bulunan `projects/team` klasörünü oluşturun. Gereksinimler: klasöre grup girebilmeli ve onu listeleyebilmeli ama değiştirememeli; `deploy.sh`'yi siz ve grup çalıştırabilmeli; `notes.txt`'yi grup okuyabilmeli; `secret.txt`'yi yalnızca siz. Diğerleri hiçbir yerde hiçbir şey alamaz. Sekizli modları verin, ayarlayın, dördünü de tek bir `stat` komutuyla gösterin ve betiği çalıştırın.

16 Tekrar Soruları

Kavramsal

1. Bir kullanıcı programı dosya izinlerini neden kolayca atlayamaz? *Sistem çağrısı* ve *çekirdek modu* kavramlarını kullanın.
2. UID ile kullanıcı adı arasındaki fark nedir? Düğüm hangisini saklar?
3. Parola özetleri neden `/etc/passwd` içinde değil de `/etc/shadow` içinde saklanır?
4. Bir dizinde r, w ve x'in anlamını açıklayın. Salt okunur bir dosyayı silmek neden mümkündür?
5. Bir dosyanın modu `----rw-rw-` ve sahibi sizsiniz. Onu okuyabilir misiniz? Kendiniz için okunabilir yapabilir misiniz?
6. Sembolik modu ne zaman, sekizli modu ne zaman kullanırsınız? Bir dizin ağacında `chmod -R 644` neden kötü bir fikirdir?
7. "Permission denied" ile "Operation not permitted" arasındaki fark nedir?
8. `passwd` neden setuid root'tur ve setuid programlar neden bir güvenlik riskidir?

Sayısal

9. Sekizliye çevirin: `rwxrwx---`, `r-xr-xr-x`, `rw-------`, `rwxr-xr--`.
10. Sembolik gösterime çevirin: 604, 751, 440, 2770.
11. Bir dosyanın modu 664. `chmod o-r,g-w` sonrasında modu nedir? Özgün moddan başlayarak `chmod a+x` sonrasında?
12. 007, 011 ve 077 umask değerleri için yeni bir dosyanın ve yeni bir dizinin modlarını hesaplayın. Bunlardan hangilerinde düz çıkarma dosya için yanlış sonuç verir?
13. Bir sınıf için kaç farklı izin bileşimi vardır? Dokuz bitin hepsi için? Üç özel bit de dâhil?

Sayısal soruların yanıtları

9. `rwxrwx---` $= 770$; `r-xr-xr-x` $= 555$; `rw-------` $= 600$; `rwxr-xr--` $= 754$.

10. 604 = `rw----r--`; 751 = `rwxr-x--x`; 440 = `r--r-----`; 2770 = `rwxrws---` (setgid `s` olarak görünür).
11. `o-r,g-w`: `rw-rw-r--` $\rightarrow$ `rw-r-----` = 640. `a+x`: `rwxrwxr-x` = 775 (her basamak $+1$, çünkü x hiçbir yerde yoktu).
12. umask 007: dosya 666 AND 770 = 660, dizin 770; çıkarma $666 - 007 = 657$ verir (yanlış). umask 011: dosya 666 (temizlenecek x yok), dizin 766; çıkarma 655 verir (yanlış). umask 077: dosya 600, dizin 700; çıkarma 567 verir (yanlış). 777 her biti içerdiğinden dizinlerde çıkarma her zaman doğrudur. (`code/perm_check.py` ile denetlendi.)
13. Bir sınıf: $2^3 = 8$. Dokuz bit: $2^9 = 512$. setuid, setgid ve yapışkan bit ile: $2^{12} = 4096$.

17 İleriye Bakış: 12. Hafta

Gelecek hafta, **Metin İşleme, Yönlendirme ve Borular**, komutları birbirine bağlıyor. Her süreç üç açık dosya tanımlayıcısıyla başlar: 0 (standart girdi), 1 (standart çıktı) ve 2 (standart hata) — 2. ve 3. haftalardaki `write(1, …)` çağrılarıyla aynı numaralar. Bunları dosyalara yönlendirecek (redirection: >, >>, 2>, 2>&1), süreçleri borularla (pipes, `|`) bağlayacak ve `~/lab/logs/access.log` web sunucusu günlüğünü `grep`, `sort`, `uniq`, `cut` ve `wc` ile çözümleyeceğiz. İzinler hemen geri dönüyor: yazamadığınız bir dosyaya çıktı yönlendirmek, bu haftanın "Permission denied" iletisinin aynısını verir.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Kısım 13.4 (Protection, File-System Interface bölümünde), Bölüm 17 (Protection) ve Bölüm 20'nin (The Linux System) güvenlik kısmı.
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Bölüm 39, "Interlude: Files and Directories" (izin bitleri ve erişim denetim listeleri kısmı): `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
  - Güvenlik bölümü "Access Control": `https://pages.cs.wisc.edu/~remzi/OSTEP/security-access.pdf`
- D. M. Ritchie ve K. Thompson, "The UNIX Time-Sharing System", *Communications of the ACM* 17(7), 1974, s. 365–375 (Kısım 3.5 koruma bitlerini ve set-user-ID bitini anlatır).

18 Laboratuvar Föyü Çözümleri

Bütün çıktılar gerçektir (`code/lab_solutions.sh`, `code/out_lab_solutions.txt` içinde yakalandı); tarihler ve numaralar sizin makinenizde farklı olacaktır.

1. `whoami; id; groups` — çıktı tam olarak Kısım 3.1 içindeki gibidir: UID 1011 (sizinki farklı olacaktır), birincil grup `student`, tek grup.

2. `grep "^$USER:" /etc/passwd` komutu `student:x:1011:1013::/home/student:/bin/bash` yazdırır. Alanlar: kullanıcı adı, parola yer

tutucusu x, UID, GID, açıklama (boş), ev dizini `/home/student`, kabuk `/bin/bash`.

3. Sahip ve grup okuyup yazabilir, diğerleri yalnızca okuyabilir: 664.

```
student@okan-lab:~/lab$ ls -l docs/syllabus.txt
-rw-rw-r-- 1 student student 578 Oct 12 19:38 docs/syllabus.txt
student@okan-lab:~/lab$ stat -c '%a  %A' docs/syllabus.txt
664  -rw-rw-r--
```

4. `rwxr-x---` $= 750$, `rw-r--r--` $= 644$, `rwx------` $= 700$; $640 =$ `rw-r-----`, $711 =$ `rwx--x--x`, $775 =$ `rwxrwxr-x`.

5. u+x yalnızca sahibe çalıştırma ekler (özgün mod 644 olsaydı `chmod 744` de kabul edilirdi; burada 664'tü).

```
student@okan-lab:~/lab$ echo '#!/bin/bash' > scripts/hello.sh
student@okan-lab:~/lab$ echo 'echo "Hello, $USER! You are in $PWD"' >> scripts/hello.sh
student@okan-lab:~/lab$ chmod u+x scripts/hello.sh
student@okan-lab:~/lab$ ls -l scripts/hello.sh
-rwxrw-r-- 1 student student 49 Oct 12 19:38 scripts/hello.sh
student@okan-lab:~/lab$ ./scripts/hello.sh
Hello, student! You are in /home/student/lab
```

6. Gruptan w'yi, diğerlerinden r'yi kaldırın (`chmod u=rw,g=r,o=` de doğrudur).

```
student@okan-lab:~/lab$ chmod g-w,o-r docs/notes_01.txt
student@okan-lab:~/lab$ ls -l docs/notes_01.txt
-rw-r----- 1 student student 16 Oct 12 19:38 docs/notes_01.txt
```

7. $600 =$ `rw-------`.

```
student@okan-lab:~/lab$ chmod 600 docs/notes_02.txt
student@okan-lab:~/lab$ stat -c '%a  %A  %n' docs/notes_02.txt
600  -rw-------  docs/notes_02.txt
```

8. 700. İçerideki dosyaların değişmesi gerekmez: `alpha`'da x olmadan başka kimse onlara ulaşamaz.

```
student@okan-lab:~/lab$ chmod 700 projects/alpha
student@okan-lab:~/lab$ ls -ld projects/alpha
drwx------ 4 student student 4096 Oct 12 19:38 projects/alpha
```

9. Dizin 750 (grup `r-x`), dosya 640. (Alt dizinler hâlâ 775'tir, ama diğerleri `beta`'yı geçemez.)

```
student@okan-lab:~/lab$ chmod 750 projects/beta
student@okan-lab:~/lab$ chmod 640 projects/beta/README.md
student@okan-lab:~/lab$ ls -ld projects/beta
drwxr-x--- 4 student student 4096 Oct 12 19:38 projects/beta
student@okan-lab:~/lab$ ls -l projects/beta
total 12
-rw-r----- 1 student student   15 Oct 12 19:38 README.md
drwxrwxr-x 2 student student 4096 Oct 12 19:38 docs
drwxrwxr-x 2 student student 4096 Oct 12 19:38 src
```

10. Tahmin: 666 AND NOT $027 = 640$, 777 AND NOT $027 = 750$.

```
student@okan-lab:~/lab$ umask 027
student@okan-lab:~/lab$ touch t10.txt
student@okan-lab:~/lab$ mkdir t10
student@okan-lab:~/lab$ stat -c '%a  %A  %n' t10.txt t10
640  -rw-r-----  t10.txt
750  drwxr-x---  t10
student@okan-lab:~/lab$ umask 002
```

11. Dosya 644, dizin 744. Bit düzeyinde: $666 = 110\,110\,110$, NOT $033 = 111\,100\,100$, AND $= 110\,100\,100 = 644$. Dosyanın hiç x biti yoktu; bu yüzden gruptan ve diğerlerinden yalnızca w kaldırılır; 633 yanlışlıkla x bitlerini "ödünç alırdı".

```
student@okan-lab:~/lab$ umask 033
student@okan-lab:~/lab$ touch t11.txt
student@okan-lab:~/lab$ mkdir t11
student@okan-lab:~/lab$ stat -c '%a  %A  %n' t11.txt t11
644  -rw-r--r--  t11.txt
744  drwxr--r--  t11
student@okan-lab:~/lab$ umask 002
```

12. `ls` adları hâlâ listeler (r var), ama `cd` ve `cat` dizinden geçmek için x'e ihtiyaç duyar.

```
student@okan-lab:~/lab$ mkdir t12
student@okan-lab:~/lab$ echo "hello" > t12/a.txt
student@okan-lab:~/lab$ chmod u-x t12
student@okan-lab:~/lab$ ls t12
a.txt
student@okan-lab:~/lab$ cd t12
bash: cd: t12: Permission denied
student@okan-lab:~/lab$ cat t12/a.txt
cat: t12/a.txt: Permission denied
student@okan-lab:~/lab$ chmod u+x t12
student@okan-lab:~/lab$ cat t12/a.txt
hello
```

13. Silmek dizini değiştirir: `docs` 775 iken salt okunur dosya (bir sorudan sonra) silinebilir. `docs` 555 iken dizin değiştirilemez, bu yüzden `rm -f` bile başarısız olur.

```
student@okan-lab:~/lab$ chmod 444 docs/notes_03.txt
student@okan-lab:~/lab$ rm docs/notes_03.txt
rm: remove write-protected regular file 'docs/notes_03.txt'? y
student@okan-lab:~/lab$ ls docs
'final report.txt'   linux_quotes.txt   notes_01.txt   notes_02.txt   syllabus.txt
student@okan-lab:~/lab$ chmod 444 docs/linux_quotes.txt
student@okan-lab:~/lab$ chmod 555 docs
student@okan-lab:~/lab$ rm -f docs/linux_quotes.txt
rm: cannot remove 'docs/linux_quotes.txt': Permission denied
student@okan-lab:~/lab$ chmod 775 docs
```

14. Çıktı Kısım 8 içindeki gibidir (burada `src/hello.c` için). `chown` root gerektirir; `chgrp students`'a başarısız olur, çünkü kullanıcı o grubun üyesi değildir. Bir yönetici `sudo chown root src/hello.c` ya da `sudo chgrp students src/hello.c` kullanırdı (ya da kullanıcıyı gruba eklerdi).

15. Çıktı Kısım 10 içindeki iki dökümle aynıdır (buradaki kendi dosya: `/tmp/w11_ex15.txt`). `s`: setuid; `passwd` root'un etkin UID'siyle çalışır, böylece `/etc/shadow`'u güncelleyebilir. `t`: yapışkan bit; `/tmp`'de bir dosyayı yalnızca sahibi silebilir.

16. Klasör 750, `deploy.sh` 750, `notes.txt` 640, `secret.txt` 600. Klasörü ve üç dosyayı 5. alıştırmadaki gibi `mkdir` ve `echo` ile oluşturun, ardından:

```
student@okan-lab:~/lab$ chmod 750 projects/team projects/team/deploy.sh
student@okan-lab:~/lab$ chmod 600 projects/team/secret.txt
student@okan-lab:~/lab$ chmod 640 projects/team/notes.txt
student@okan-lab:~/lab$ stat -c '%a  %A  %n' projects/team projects/team/*
750  drwxr-x---  projects/team
750  -rwxr-x---  projects/team/deploy.sh
640  -rw-r-----  projects/team/notes.txt
600  -rw-------  projects/team/secret.txt
student@okan-lab:~/lab$ ./projects/team/deploy.sh
deploying...
```